Number: P3771R0
Date: 2025-06-30
Audience: SG1: Concurrency, Library Evolution, and Evolution
Target: C++29
Author: Hana Dusíková

Thanks to Michal Tichák, and Tomasz Kamiński for providing feedback, help, and encouragement.

P3771R0: constexpr mutex, locks, and condition v

Motivation

It's really hard to conditionally avoid non-`constexpr` types in a code which is supposed to be `constexpr` compatible. This paper fixes it by making these types (and algorithms) `constexpr` compatible. There is no semantical change for updated types and algorithms.

This paper is a continuation of paper P3309R3: `constexpr atomic & atomic_ref` and makes a lot of library code reusable in `constexpr` world.

Reusability

Main objective is being able to reuse same code in any environment (runtime, GPU, and now also constant evaluated). This makes C++ programs much easier to read and write and more bug-prone as someone wise once said *"for every line there is a bug"*.

Also it lowers cognitive burden when users don't need to care what is and what's not `constexpr` compatible.

Example with `std::mutex`

Type `std::mutex` has `constexpr` default constructor since C++11, but `std::lock_guard` doesn't. You can't just wrap the `auto _ = std::lock_guard{mtx}` into `if consteval`.

```
template <typename T> class locked_queue {
  std::queue<T> impl{};
  std::mutex mtx{};
public:
  locked_queue() = default;

  constexpr void push(T input) {
    auto _ = std::lock_guard{mtx}; // BEFORE: unable to call non-constexpr constru
ctor
                                   // AFTER: fine
    impl.push(std::move(input));
  }
  constexpr std::optional<T> pop() {
    auto _ = std::lock_guard{mtx}; // BEFORE: unable to call non-constexpr constru
ctor
                                   // AFTER: fine
    if (impl.empty()) {
      return std::nullopt;
    }

    auto r = std::move(impl.front());
    impl.pop();
    return r;
  }
}

consteval foo my_algorithm() {
  auto queue = locked_queue<int>{};
  queue.push(1);  // BEFORE: unable to call non-constexpr constructor of `lock_guar
d`
  queue.push(42); //
  queue.push(11); //
                  // AFTER: I can reuse my code!
  return process(queue);
}
```

One possibility to make this work without this paper would look like this:

```
template <typename T> class locked_queue {
  std::queue<T> impl{};
  std::mutex mtx{};

  constexpr void unsync_push(T input) {
    impl.push(std::move(input));
  }

  constexpr std::optional<T> unsync_pop() {
    if (impl.empty()) {
      return std::nullopt;
    }

    auto r = std::move(impl.front());
    impl.pop();
    return r;
  }

public:
  constexpr void push(T input) {
    if consteval {
      unsafe_push(std::move(input));
    } else {
      auto _ = std::lock_guard{mtx};
      unsafe_push(std::move(input));
    }
  }
  constexpr std::optional<T> pop() {
    if consteval {
      return unsafe_pop();
    } else {
      auto _ = std::lock_guard{mtx};
      return unsafe_pop();
    }
  }
}
```

This pattern is terrible and it creates opportunities for more bugs, and it makes testing harder (especially when test coverage mapping is used).

Example with `std::shared_mutex`

```
template <typename T> class protected_object {
  std::optional<T> object{std::nullopt};
  std::shared_mutex mtx{}; // BEFORE: unable to use non-constexpr constructor
                           // AFTER: fine
public:
  template <typename... Args> constexpr void set(Args && ... args) {
    auto _ = std::unique_lock{mtx};
    object.emplace(std::forward<Args>(args)...);
  }
  std::optional<T;> get() const {
    auto _ = std::shared_lock{mtx};
    return object;
  }
}
```

Type `std::shared_mutex` doesn't have `constexpr` default constructor. There is not a simple solution how to avoid the error.

No deadlocks

A deadlock is undefined behaviour (missing citation). Any undefined behaviour during constant evaluation is making program ill-formed [expr.const].

Change

In terms of specification it's just adding `constexpr` everywhere in section thread.mutex (including thread.lock and thread.once) and thread.condition. Semantic of all types and algorithms is clearly defined and is non-surprising to users.

One optional additional wording change is making sure no synchronization primitive can leave constant evaluation in non-default state (which will be useful also in future for `semaphors`).

Quesion about timed locking

Types `timed_mutex`, `shared_timed_mutex`, `recursive_timed_mutex`, and some methods on `unique_lock` and `shared_lock` have functionality which allows to give up and not take the ownership of lock after certain time or at specific timepoint. There is not observable time during constant evaluation, there are three possible options:

- simply not make `try_lock_for` nor `try_lock_until` functionality `constexpr` (and forcing users to `if consteval` such code away, which is against motivation of this paper),
- automatically fail to take ownership if already locked (author's preferred, with wording),

- make only `try_lock_for` `constexpr`, and not make `try_lock_until`, and block compilation for specified duration.

I prefer option with quick failure to take the ownership, as we know in single-threaded environment we would block for some time and fail anyway. And there is no way how to observe time during constant evaluation anyway. You can think about this as fast forward of time.

Native handles

Functions returning native handles are not marked `constexpr` as these are an escape hatch to use platform specific code.

Utility functions

There is function `notify_all_at_thread_exit` which is marked `constexpr`, in single threaded environment is a no-op and it's trivial to implement it that way. If we won't implement it, we will force users to write `if consteval` everytime they use it, and that's not a good user experience.

This paper also proposes making `constexpr` free functions implementing interruptable waits, and we can do so as `stop_token` is already `constexpr` default constructible thanks to making `shared_ptr` constexpr in P3037R6. Because there is no other thread which can interrupt the wait, these function will behave similarly as non-interruptable wait functions (meaning fail immediately to obtain lock due the new paragraph in [thread.req.timing]).

Implementation

I started working on the implementation in libc++, following subsections contains notes from my research how all three major standard library implements impacted types from this paper.

libc++

Type `mutex` and other mutex-based types has definition of methods in a source file in which these methods are abstract into low-level `__libcpp_mutex_*` functions, which are defined in their platform specific header files (dispatched here, C11, POSIX, win32 with its implementation). Support to `constexpr` mutex default constructor is done thru providing constant `_LIBCPP_MUTEX_INITIALIZER` which is defined as `{}` (for C11) or `PTHREAD_MUTEX_INITIALIZER` (for POSIX threads). This tells me same thing (creating `_LIBCPP_*_INITIALIZER` macros) can be done for other mutex types, as this is already supported by posix threads and can be done also with win32.

Type `condition_variable` default constructor is surprisingly already constexpr (so much about the requirement in [constexpr.function]). Main functionality is in a source file where it is using already abstracted away functions similarly as mutex. These few files will need to be moved to header file too.

why is it in source files

Based on my experience implementing `constexpr` exceptions I have noticed libc++ tends to hide the exception throwing code in to source files in shared library. This code mostly will be needed to moved to header files anyway due the `constexpr` exception support. The two layer of abstraction for `mutex` and `condition_variable` won't be needed anymore after that.

`shared_mutex` implementation detail in libc++

This is code from libc++ with removed annotational macros. Unfortunetely the `__shared_mutex_base` contains methods defined in a `.cpp` file. But internal `mutex` and `condition_variable` types are already default constexpr constructible and the `__shared_mutex_base` constructor only sets `__state_` to zero, so this constructor can be easily made `constexpr`.

All the single-threaded semantic can be then put into `shared_mutex` type, with `if consteval` and compiler builtin to attach constant evaluation metadata to an object.

```
struct __shared_mutex_base {
  mutex __mut_;
  condition_variable __gate1_;
  condition_variable __gate2_;
  unsigned __state_{0};

  static const unsigned __write_entered_ = 1U << (sizeof(unsigned) * __CHAR_BIT__
- 1);
  static const unsigned __n_readers_     = ~__write_entered_;

  __shared_mutex_base() = default;
  ~__shared_mutex_base() = default;

  __shared_mutex_base(const __shared_mutex_base&)             = delete;
  __shared_mutex_base& operator=(const __shared_mutex_base&) = delete;

  // Exclusive ownership
  void lock(); // blocking
  bool try_lock();
  void unlock();

  // Shared ownership
  void lock_shared(); // blocking
  bool try_lock_shared();
  void unlock_shared();
};

class shared_mutex {
  __shared_mutex_base __base_;

public:
  constexpr shared_mutex() : __base_() {}
  ~shared_mutex() = default;

  shared_mutex(const shared_mutex&)             = delete;
  shared_mutex& operator=(const shared_mutex&) = delete;

  // Exclusive ownership
  constexpr void lock() {
    if consteval {
      return __builtin_metadata_unique_lock(&__base_);
      if (__base_.state != 0) std::abort;
      __base_.state = 1;
    } else {
      return __base_.lock();
    }
  }
  constexpr bool try_lock() {
    if consteval {
      return __builtin_metadata_try_unique_lock(&__base_);
```

```
      if (__base_.state != 0) return false;
      __base_.state = 1;
      return true;
    } else {
      return __base_.try_lock();
    }
  }
  constexpr void unlock() {
    if consteval {
      return __builtin_metadata_unique_unlock(&__base_);
      if (__base_.state != 1) std::abort();
      __base_.state = 0;
    } else {
      return __base_.unlock();
    }
  }

  // Shared ownership
  constexpr void lock_shared() {
    if consteval {
      return __builtin_metadata_shared_lock(&__base_);
      if (__base_.state == 0) __base_.state = 2;
      else if (__base_.state == 1) std::abort();
      else ++__base_.state;
    } else {
      return __base_.lock_shared();
    }
  }
  constexpr bool try_lock_shared() {
    if consteval {
      return __builtin_metadata_try_shared_lock(&__base_);
      if (__base_.state == 0) __base_.state = 2;
      else if (__base_.state == 1) return false;
      else ++__base_.state;
      return true;
    } else {
      return __base_.try_lock_shared();
    }
  }
  constexpr void unlock_shared() {
    if consteval {
      return __builtin_metadata_shared_unlock(&__base_);
      if (__base_.state == 0) std::abort();
      else if (__base_.state == 1) std::abort();
     else if (__base_.state == 2) __base_.state = 0;
      else --__base_.state;
    } else {
      return __base_.unlock_shared();
    }
```

```
  }
};
```

Purposes of these builtins is to provided associated metadata to the object itself, and use them for useful diagnostics.

Alternative approach would be use `__shared_mutex_base::__state_` for it, and use library functionality to provide error messages in case of deadlock, but this approach doesn't allow us easily to diagnose where the lock was previously obtained.

Research of libstdc++

Libstdc++ is supporting many platforms and code around synchronization primitives is somehow bit convoluted due macro abstractions and `#ifdef`-s rules to select appropriate implementation.

std::mutex

Most basic mutex type is all defined in headers it's default constructor is defaulted or explicitly made constexpr in presence of macro `__GTHREAD_MUTEX_INIT`. Native type is hidden in `__mutex_base` which in order to support constexpr *default* initialize the `__gthread_mutex_t` native handle.

std::shared_mutex

Shared mutex type's default constructor is not historically marked `constexpr`. It's also defined all in a header file. Construction of native handle is abstract in `__shared_mutex_pthread` or `__shared_mutex_cv` which is used when platform doesn't provide `_GLIBCXX_USE_PTHREAD_RWLOCK_T` and its implementation is normal mutex and two condition variables, similarly as in libc++.

In case pthread of the platform provides RW lock, the type `__shared_mutex_pthread` abstracts it away. If macro `PTHREAD_RWLOCK_INITIALIZER` is available, it's used to defualt initialize the lock. This macro is on most major platform providing aggregate or numeric constant initialization (macOS, linux, win32's pthreads).

lock guards

All lock guards types (`lock_guard`, `unique_lock`, `shared_lock`, `scoped_lock`) are just RAII wrappers over interfaces of mutexes. There is nothing special and nothing which interfere with making them constexpr, these are already defined in header files.

std::condition_variable

Condition variable types are defined partially in header file. Implementation of some function is in a source file and they are not doing anything platform specific. Type `__condvar` which abstracts native handle is defined next to std::mutex. It doesn't contain same abstraction as was done for the mutex type, even when most pthread libraries provides equivalent macro `PTHREAD_COND_INITIALIZER` for constant initialization. This piece of code would benefit from update. Making `__condvar` `constexpr` compatible would be then trivial.

MS STL

`mutex`, `recursive_mutex`, and `timed_mutex` all share `_Mutex_base` and making these types `constexpr` will be trivial with `if consteval`. Same applies to `shared_mutex` too.

Type `condition_variable` is defined in header, but it uses implementation function defined in source files. These can be circumvent easily with `if consteval`. There is a `_DISABLE_CONSTEXPR_MUTEX_CONSTRUCTOR` macro, which optionaly disables `std::mutex`'s constructor and it makes the condition variable initialized with function defined outside. This is a deviation from the standard, and when this special macro is not set there is nothing which prevents condition variable to be `constexpr` compatible.

Wording

Modify constant evaluation

Purpose of this change is to disallow creation of already locked synchronization objects and their subsequent leakage into runtime code.

7.7 Constant expressions [expr.const]

10 An expression *E* is a *core constant expression* unless the evaluation of *E*, following the rules of the abstract machine ([intro.execution]), would evaluate one of the following:

(10.1) — `this` ([expr.prim.this]), except

(10.1.1) — in a constexpr function ([dcl.constexpr]) that is being evaluated as part of *E* or

(10.1.2) — when appearing as the *postfix-expression* of an implicit or explicit class member access expression ([expr.ref]);

(10.2) — a control flow that passes through a declaration of a block variable ([basic.scope.block]) with static ([basic.stc.static]) or thread ([basic.stc.thread]) storage duration, unless that variable is usable in constant expressions;

[*Example 4*:

```
constexpr char test() {
  static const int x = 5;
```

```
  static constexpr char c[] = "Hello World";
  return *(c + x);
}
static_assert(' ' == test());
```

— *end example*]

(10.3) — an invocation of a non-constexpr function;

(10.4) — an invocation of an undefined constexpr function;

(10.5) — an invocation of an instantiated constexpr function that is not constexpr-suitable;

(10.6) — an invocation of a virtual function ([class.virtual]) for an object whose dynamic type is constexpr-unknown;

(10.7) — an expression that would exceed the implementation-defined limits (see [implimits]);

(10.8) — an operation that would have undefined or erroneous behavior as specified in [intro] through [cpp];

(10.9) — an lvalue-to-rvalue conversion unless it is applied to

(10.9.1) — a glvalue of type *cv* `std::nullptr_t`,

(10.9.2) — a non-volatile glvalue that refers to an object that is usable in constant expressions, or

(10.9.3) — a non-volatile glvalue of literal type that refers to a non-volatile object whose lifetime began within the evaluation of *E*;

(10.10) — an lvalue-to-rvalue conversion that is applied to a glvalue that refers to a non-active member of a union or a subobject thereof;

(10.11) — an lvalue-to-rvalue conversion that is applied to an object with an indeterminate value;

(10.12) — an invocation of an implicitly-defined copy/move constructor or copy/move assignment operator for a union whose active member (if any) is mutable, unless the lifetime of the union object began within the evaluation of *E*;

(10.13) — in a *lambda-expression*, a reference to `this` or to a variable with automatic storage duration defined outside that *lambda-expression*, where the reference would be an odr-use ([basic.def.odr], [expr.prim.lambda]);

[*Example 5*:

```
void g() {
  const int n = 0;
  [=] {
    constexpr int i = n;          // OK, n is not odr-used here
    constexpr int j = *&n;        // error: &n would be an odr-use of n
  };
}
```

— *end example*]

[*Note 4*: If the odr-use occurs in an invocation of a function call operator of a closure type, it no longer refers to `this` or to an enclosing variable with automatic storage duration due to the transformation ([expr.prim.lambda.capture]) of the *id-expression* into an access of the corresponding data member.

[*Example 6*:

```
auto monad = [](auto v) { return [=] { return v; }; };
auto bind = [](auto m) {
  return [=](auto fvm) { return fvm(m()); };
};

// OK to capture objects with automatic storage duration created during constant expre
ssion evaluation.
static_assert(bind(monad(2))(monad)() == monad(2)());
```

— *end example*]

— *end note*]

(10.14) — a conversion from a prvalue `P` of type "pointer to *cv* `void`" to a type "*cv1* pointer to `T`", where `T` is not *cv2* `void`, unless `P` is a null pointer value or points to an object whose type is similar to `T`;

(10.15) — a `reinterpret_cast` ([expr.reinterpret.cast]);

(10.16) — a modification of an object ([expr.assign], [expr.post.incr], [expr.pre.incr]) unless it is applied to a non-volatile lvalue of literal type that refers to a non-volatile object whose lifetime began within the evaluation of *E*;

(10.17) — an invocation of a destructor ([class.dtor]) or a function call whose *postfix-expression* names a pseudo-destructor ([expr.call]), in either case for an object whose lifetime did not begin within the evaluation of *E*;

(10.18) — a *new-expression* ([expr.new]), unless either

(10.18.1) — the selected allocation function is a replaceable global allocation function ([new.delete.single], [new.delete.array]) and the allocated storage is deallocated within the evaluation of *E*, or

(10.18.2) — the selected allocation function is a non-allocating form ([new.delete.placement]) with an allocated type `T`, where

(10.18.2.1) — the placement argument to the *new-expression* points to an object whose type is similar to `T` ([conv.qual]) or, if `T` is an array type, to the first element of an object of a type similar to `T`, and

(10.18.2.2) — the placement argument points to storage whose duration began within the evaluation of *E*;

(10.19) — a *delete-expression* ([expr.delete]), unless it deallocates a region of storage allocated within the evaluation of *E*;

(10.20)

— a call to an instance of `std::allocator<T>::allocate` ([allocator.members]), unless the allocated storage is deallocated within the evaluation of *E*;

(10.21) — a call to an instance of `std::allocator<T>::deallocate` ([allocator.members]), unless it deallocates a region of storage allocated within the evaluation of *E*;

(10.22) — a construction of an exception object, unless the exception object and all of its implicit copies created by invocations of `std::current_exception` or `std::rethrow_exception` ([propagation]) are destroyed within the evaluation of *E*;

(10.??) — a construction of a synchronization object [thread], unless the object is destroyed within the evaluation of *E* or is in its default state when evaluation of *E* is finished;

(10.23) — an *await-expression* ([expr.await]);

(10.24) — a *yield-expression* ([expr.yield]);

(10.25) — a three-way comparison ([expr.spaceship]), relational ([expr.rel]), or equality ([expr.eq]) operator where the result is unspecified;

(10.26) — a `dynamic_cast` ([expr.dynamic.cast]) or `typeid` ([expr.typeid]) expression on a glvalue that refers to an object whose dynamic type is constexpr-unknown;

(10.27) — a `dynamic_cast` ([expr.dynamic.cast]) expression, `typeid` ([expr.typeid]) expression, or `new`-`expression` ([expr.new]) that would throw an exception where no definition of the exception type is reachable;

(10.28) — an *asm-declaration* ([dcl.asm]);

(10.29) — an invocation of the `va_arg` macro ([cstdarg.syn]);

(10.30) — a non-constant library call ([defns.nonconst.libcall]); or

(10.31) — a `goto` statement ([stmt.goto]).

[*Note 5*: A `goto` statement introduced by equivalence ([stmt]) is not in scope. For example, a `while` statement ([stmt.while]) can be executed during constant evaluation. — *end note*]

Mutexes and locking

32.6 Mutual exclusion [thread.mutex]

32.6.1 General [thread.mutex.general]

1 Subclause [thread.mutex] provides mechanisms for mutual exclusion: mutexes, locks, and call once. These mechanisms ease the production of race-free programs ([intro.multithread]).

32.6.2 Header <mutex> synopsis [mutex.syn]

```
namespace std {
  // [thread.mutex.class], class mutex
  class mutex;
  // [thread.mutex.recursive], class recursive_mutex
  class recursive_mutex;
  // [thread.timedmutex.class], class timed_mutex
  class timed_mutex;
  // [thread.timedmutex.recursive], class recursive_timed_mutex
  class recursive_timed_mutex;

  struct defer_lock_t { explicit defer_lock_t() = default; };
  struct try_to_lock_t { explicit try_to_lock_t() = default; };
  struct adopt_lock_t { explicit adopt_lock_t() = default; };

  inline constexpr defer_lock_t  defer_lock { };
  inline constexpr try_to_lock_t try_to_lock { };
  inline constexpr adopt_lock_t  adopt_lock { };

  // [thread.lock], locks
  template<class Mutex> class lock_guard;
  template<class... MutexTypes> class scoped_lock;
  template<class Mutex> class unique_lock;

  template<class Mutex>
    constexpr void swap(unique_lock<Mutex>& x, unique_lock<Mutex>&
y) noexcept;

  // [thread.lock.algorithm], generic locking algorithms
  template<class L1, class L2, class... L3> int try_lock(L1&, L2&, L3
&...);
  template<class L1, class L2, class... L3> void lock(L1&, L2&, L3
&...);

  struct once_flag;

  template<class Callable, class... Args>
    constexpr void call_once(once_flag& flag, Callable&& func, Args&
&... args);
}
```

32.6.3 Header <shared_mutex> synopsis [shared.mutex.syn]

```
namespace std {
  // [thread.sharedmutex.class], class shared_mutex
  class shared_mutex;
  // [thread.sharedtimedmutex.class], class shared_timed_mutex
  class shared_timed_mutex;
  // [thread.lock.shared], class template shared_lock
  template<class Mutex> class shared_lock;
  template<class Mutex>
    constexpr void swap(shared_lock<Mutex>& x, shared_lock<Mutex>&
y) noexcept;
}
```

32.6.4 Mutex requirements [thread.mutex.requirements]

32.6.4.1 General [thread.mutex.requirements.general]

1 A mutex object facilitates protection against data races and allows safe synchronization of data between execution agents. An execution agent *owns* a mutex from the time it successfully calls one of the lock functions until it calls unlock. Mutexes can be either recursive or non-recursive, and can grant simultaneous ownership to one or many execution agents. Both recursive and non-recursive mutexes are supplied.

32.6.4.2 Mutex types [thread.mutex.requirements.mutex]

32.6.4.2.1 General [thread.mutex.requirements.mutex.general]

1 The *mutex types* are the standard library types `mutex`, `recursive_mutex`, `timed_mutex`, `recursive_timed_mutex`, `shared_mutex`, and `shared_timed_mutex`. They meet the requirements set out in [thread.mutex.requirements.mutex]. In this description, `m` denotes an object of a mutex type.

[*Note 1*: The mutex types meet the *Cpp17Lockable* requirements ([thread.req.lockable.req]). — *end note*]

2 The mutex types meet *Cpp17DefaultConstructible* and *Cpp17Destructible*. If initialization of an object of a mutex type fails, an exception of type `system_error` is thrown. The mutex types are neither copyable nor movable.

3 The error conditions for error codes, if any, reported by member functions of the mutex types are as follows:

(3.1) — `resource_unavailable_try_again` — if any native handle type manipulated is not available.

(3.2) — `operation_not_permitted` — if the thread does not have the privilege to perform the operation.

(3.3) —

`invalid_argument` — if any native handle type manipulated as part of mutex construction is incorrect.

4 The implementation provides lock and unlock operations, as described below. For purposes of determining the existence of a data race, these behave as atomic operations ([intro.multithread]). The lock and unlock operations on a single mutex appears to occur in a single total order.

[*Note 2*: This can be viewed as the modification order of the mutex. — *end note*]

[*Note 3*: Construction and destruction of an object of a mutex type need not be thread-safe; other synchronization can be used to ensure that mutex objects are initialized and visible to other threads. — *end note*]

5 The expression `m.lock()` is well-formed and has the following semantics:

6 *Preconditions*: If `m` is of type `mutex`, `timed_mutex`, `shared_mutex`, or `shared_timed_mutex`, the calling thread does not own the mutex.

7 *Effects*: Blocks the calling thread until ownership of the mutex can be obtained for the calling thread.

8 *Synchronization*: Prior `unlock()` operations on the same object *synchronize with* ([intro.multithread]) this operation.

9 *Postconditions*: The calling thread owns the mutex.

10 *Return type*: `void`.

11 *Throws*: `system_error` when an exception is required ([thread.req.exception]).

12 *Error conditions*:

(12.1) — `operation_not_permitted` — if the thread does not have the privilege to perform the operation.

(12.2) — `resource_deadlock_would_occur` — if the implementation detects that a deadlock would occur.

13 The expression `m.try_lock()` is well-formed and has the following semantics:

14 *Preconditions*: If `m` is of type `mutex`, `timed_mutex`, `shared_mutex`, or `shared_timed_mutex`, the calling thread does not own the mutex.

15 *Effects*: Attempts to obtain ownership of the mutex for the calling thread without blocking. If ownership is not obtained, there is no effect and `try_lock()` immediately returns. An implementation may fail to obtain the lock even if it is not held by any other thread.

[*Note 4*: This spurious failure is normally uncommon, but allows interesting implementations based on a simple compare and exchange ([atomics]). — *end note*]

An implementation should ensure that `try_lock()` does not consistently return `false` in the absence of contending mutex acquisitions.

Synchronization: If `try_lock()` returns `true`, prior `unlock()` operations on the same object synchronize with this operation.

[*Note 5*: Since `lock()` does not synchronize with a failed subsequent `try_lock()`, the visibility rules are weak enough that little would be known about the state after a failure, even in the absence of spurious failures. — *end note*]

Return type: `bool`.

Returns: `true` if ownership was obtained, otherwise `false`.

Throws: Nothing.

The expression `m.unlock()` is well-formed and has the following semantics:

Preconditions: The calling thread owns the mutex.

Effects: Releases the calling thread's ownership of the mutex.

Return type: `void`.

Synchronization: This operation synchronizes with subsequent lock operations that obtain ownership on the same object.

Throws: Nothing.

32.6.4.2.2 Class `mutex` [thread.mutex.class]

```
namespace std {
  class mutex {
  public:
    constexpr mutex() noexcept;
    constexpr ~mutex();

    mutex(const mutex&) = delete;
    mutex& operator=(const mutex&) = delete;

    constexpr void lock();
    constexpr bool try_lock();
    constexpr void unlock();

    using native_handle_type = implementation-defined;          // see [thread.req.native]
    native_handle_type native_handle();                         // see [thread.req.native]
  };
}
```

The class `mutex` provides a non-recursive mutex with exclusive ownership semantics. If one thread owns a mutex object, attempts by another thread to acquire ownership of that object will fail (for `try_lock()`) or block (for `lock()`) until the owning thread has released ownership with a call to `unlock()`.

[3] The class `mutex` meets all of the mutex requirements ([thread.mutex.requirements]). It is a standard-layout class ([class.prop]).

[5] The behavior of a program is undefined if it destroys a `mutex` object owned by any thread or a thread terminates while owning a `mutex` object.

32.6.4.2.3 Class `recursive_mutex` [thread.mutex.recursive]

```
namespace std {
  class recursive_mutex {
  public:
    constexpr recursive_mutex();
    constexpr ~recursive_mutex();

    recursive_mutex(const recursive_mutex&) = delete;
    recursive_mutex& operator=(const recursive_mutex&) = delete;

    constexpr void lock();
    constexpr bool try_lock() noexcept;
    constexpr void unlock();

    using native_handle_type = implementation-defined;          // see [thread.req.native]
    native_handle_type native_handle();                         // see [thread.req.native]
  };
}
```

[1] The class `recursive_mutex` provides a recursive mutex with exclusive ownership semantics. If one thread owns a `recursive_mutex` object, attempts by another thread to acquire ownership of that object will fail (for `try_lock()`) or block (for `lock()`) until the first thread has completely released ownership.

[2] The class `recursive_mutex` meets all of the mutex requirements ([thread.mutex.requirements]). It is a standard-layout class ([class.prop]).

[3] A thread that owns a `recursive_mutex` object may acquire additional levels of ownership by calling `lock()` or `try_lock()` on that object. It is unspecified how many levels of ownership may be acquired by a single thread. If a thread has already acquired the maximum level of ownership for a `recursive_mutex` object, additional calls to `try_lock()` fail, and additional calls to `lock()` throw an exception

of type `system_error`. A thread shall call `unlock()` once for each level of ownership acquired by calls to `lock()` and `try_lock()`. Only when all levels of ownership have been released may ownership be acquired by another thread.

4 The behavior of a program is undefined if

(4.1) — it destroys a `recursive_mutex` object owned by any thread or

(4.2) — a thread terminates while owning a `recursive_mutex` object.

32.6.4.3 Timed mutex types [thread.timedmutex.requirements]

32.6.4.3.1 General [thread.timedmutex.requirements.general]

1 The *timed mutex types* are the standard library types `timed_mutex`, `recursive_timed_mutex`, and `shared_timed_mutex`. They meet the requirements set out below. In this description, `m` denotes an object of a mutex type, `rel_time` denotes an object of an instantiation of `duration`, and `abs_time` denotes an object of an instantiation of `time_point`.

[*Note 1*: The timed mutex types meet the *Cpp17TimedLockable* requirements ([thread.req.lockable.timed]). — *end note*]

2 The expression `m.try_lock_for(rel_time)` is well-formed and has the following semantics:

3 *Preconditions*: If `m` is of type `timed_mutex` or `shared_timed_mutex`, the calling thread does not own the mutex.

4 *Effects*: The function attempts to obtain ownership of the mutex within the relative timeout ([thread.req.timing]) specified by `rel_time`. If the time specified by `rel_time` is less than or equal to `rel_time.zero()`, the function attempts to obtain ownership without blocking (as if by calling `try_lock()`). The function returns within the timeout specified by `rel_time` only if it has obtained ownership of the mutex object.

[*Note 2*: As with `try_lock()`, there is no guarantee that ownership will be obtained if the lock is available, but implementations are expected to make a strong effort to do so. — *end note*]

5 *Synchronization*: If `try_lock_for()` returns `true`, prior `unlock()` operations on the same object *synchronize with* ([intro.multithread]) this operation.

6 *Return type*: `bool`.

7 *Returns*: `true` if ownership was obtained, otherwise `false`.

8 *Throws*: Timeout-related exceptions ([thread.req.timing]).

9

The expression `m.try_lock_until(abs_time)` is well-formed and has the following semantics:

10 *Preconditions*: If `m` is of type `timed_mutex` or `shared_timed_mutex`, the calling thread does not own the mutex.

11 *Effects*: The function attempts to obtain ownership of the mutex. If `abs_time` has already passed, the function attempts to obtain ownership without blocking (as if by calling `try_lock()`). The function returns before the absolute timeout ([thread.req.timing]) specified by `abs_time` only if it has obtained ownership of the mutex object.

[*Note 3*: As with `try_lock()`, there is no guarantee that ownership will be obtained if the lock is available, but implementations are expected to make a strong effort to do so. — *end note*]

12 *Synchronization*: If `try_lock_until()` returns `true`, prior `unlock()` operations on the same object *synchronize with* ([intro.multithread]) this operation.

13 *Return type*: `bool`.

14 *Returns*: `true` if ownership was obtained, otherwise `false`.

15 *Throws*: Timeout-related exceptions ([thread.req.timing]).

32.6.4.3.2 Class `timed_mutex` [thread.timedmutex.class]

```
namespace std {
  class timed_mutex {
  public:
    constexpr timed_mutex();
    constexpr ~timed_mutex();

    timed_mutex(const timed_mutex&) = delete;
    timed_mutex& operator=(const timed_mutex&) = delete;

    constexpr void lock();     // blocking
    constexpr bool try_lock();
    template<class Rep, class Period>
      constexpr bool try_lock_for(const chrono::duration<Rep, Period>
& rel_time);
    template<class Clock, class Duration>
      constexpr bool try_lock_until(const chrono::time_point<Clock, D
uration>& abs_time);
    constexpr void unlock();

    using native_handle_type = implementation-defined;          // see [t
hread.req.native]
```

```
    native_handle_type native_handle();                    // see
[thread.req.native]
  };
}
```

1 The class `timed_mutex` provides a non-recursive mutex with exclusive ownership semantics. If one thread owns a `timed_mutex` object, attempts by another thread to acquire ownership of that object will fail (for `try_lock()`) or block (for `lock()`, `try_lock_for()`, and `try_lock_until()`) until the owning thread has released ownership with a call to `unlock()` or the call to `try_lock_for()` or `try_lock_until()` times out (having failed to obtain ownership).

2 The class `timed_mutex` meets all of the timed mutex requirements ([thread.timedmutex.requirements]). It is a standard-layout class ([class.prop]).

3 The behavior of a program is undefined if

(3.1) — it destroys a `timed_mutex` object owned by any thread,

(3.2) — a thread that owns a `timed_mutex` object calls `lock()`, `try_lock()`, `try_lock_for()`, or `try_lock_until()` on that object, or

(3.3) — a thread terminates while owning a `timed_mutex` object.

32.6.4.3.3 Class `recursive_timed_mutex` [thread.timedmutex.recursive]

```
namespace std {
  class recursive_timed_mutex {
  public:
    constexpr recursive_timed_mutex();
    constexpr ~recursive_timed_mutex();

    recursive_timed_mutex(const recursive_timed_mutex&) = delete;
    recursive_timed_mutex& operator=(const recursive_timed_mutex&) =
delete;

    constexpr void lock();     // blocking
    constexpr bool try_lock() noexcept;
    template<class Rep, class Period>
      constexpr bool try_lock_for(const chrono::duration<Rep, Period>
& rel_time);
    template<class Clock, class Duration>
      constexpr bool try_lock_until(const chrono::time_point<Clock, D
uration>& abs_time);
    constexpr void unlock();

    using native_handle_type = implementation-defined;          // see [t
hread.req.native]
    native_handle_type native_handle();                    // see
```

[thread.req.native]

```
  };
}
```

1 The class `recursive_timed_mutex` provides a recursive mutex with exclusive ownership semantics. If one thread owns a `recursive_timed_mutex` object, attempts by another thread to acquire ownership of that object will fail (for `try_lock()`) or block (for `lock()`, `try_lock_for()`, and `try_lock_until()`) until the owning thread has completely released ownership or the call to `try_lock_for()` or `try_lock_until()` times out (having failed to obtain ownership).

2 The class `recursive_timed_mutex` meets all of the timed mutex requirements ([thread.timedmutex.requirements]). It is a standard-layout class ([class.prop]).

3 A thread that owns a `recursive_timed_mutex` object may acquire additional levels of ownership by calling `lock()`, `try_lock()`, `try_lock_for()`, or `try_lock_until()` on that object. It is unspecified how many levels of ownership may be acquired by a single thread. If a thread has already acquired the maximum level of ownership for a `recursive_timed_mutex` object, additional calls to `try_lock()`, `try_lock_for()`, or `try_lock_until()` fail, and additional calls to `lock()` throw an exception of type `system_error`. A thread shall call `unlock()` once for each level of ownership acquired by calls to `lock()`, `try_lock()`, `try_lock_for()`, and `try_lock_until()`. Only when all levels of ownership have been released may ownership of the object be acquired by another thread.

4 The behavior of a program is undefined if

(4.1) — it destroys a `recursive_timed_mutex` object owned by any thread, or

(4.2) — a thread terminates while owning a `recursive_timed_mutex` object.

32.6.4.4 Shared mutex types [thread.sharedmutex.requirements]

32.6.4.4.1 General [thread.sharedmutex.requirements.general]

1 The standard library types `shared_mutex` and `shared_timed_mutex` are *shared mutex types*. Shared mutex types meet the requirements of mutex types ([thread.mutex.requirements.mutex]) and additionally meet the requirements set out below. In this description, `m` denotes an object of a shared mutex type.

[*Note 1*: The shared mutex types meet the *Cpp17SharedLockable* requirements ([thread.req.lockable.shared]). — *end note*]

2 In addition to the exclusive lock ownership mode specified in [thread.mutex.requirements.mutex], shared mutex types provide a *shared lock* ownership mode. Multiple execution agents can simultaneously hold a shared lock ownership of a shared mutex type. But no execution agent holds a shared lock while another execu-

tion agent holds an exclusive lock on the same shared mutex type, and vice-versa. The maximum number of execution agents which can share a shared lock on a single shared mutex type is unspecified, but is at least 10000. If more than the maximum number of execution agents attempt to obtain a shared lock, the excess execution agents block until the number of shared locks are reduced below the maximum amount by other execution agents releasing their shared lock.

3 The expression `m.lock_shared()` is well-formed and has the following semantics:

4 *Preconditions*: The calling thread has no ownership of the mutex.

5 *Effects*: Blocks the calling thread until shared ownership of the mutex can be obtained for the calling thread. If an exception is thrown then a shared lock has not been acquired for the current thread.

6 *Synchronization*: Prior `unlock()` operations on the same object synchronize with ([intro.multithread]) this operation.

7 *Postconditions*: The calling thread has a shared lock on the mutex.

8 *Return type*: `void`.

9 *Throws*: `system_error` when an exception is required ([thread.req.exception]).

10 *Error conditions*:

(10.1) — `operation_not_permitted` — if the thread does not have the privilege to perform the operation.

(10.2) — `resource_deadlock_would_occur` — if the implementation detects that a deadlock would occur.

11 The expression `m.unlock_shared()` is well-formed and has the following semantics:

12 *Preconditions*: The calling thread holds a shared lock on the mutex.

13 *Effects*: Releases a shared lock on the mutex held by the calling thread.

14 *Return type*: `void`.

15 *Synchronization*: This operation synchronizes with subsequent `lock()` operations that obtain ownership on the same object.

16 *Throws*: Nothing.

17 The expression `m.try_lock_shared()` is well-formed and has the following semantics:

18 *Preconditions*: The calling thread has no ownership of the mutex.

19

Effects: Attempts to obtain shared ownership of the mutex for the calling thread without blocking. If shared ownership is not obtained, there is no effect and `try_lock_shared()` immediately returns. An implementation may fail to obtain the lock even if it is not held by any other thread.

20 *Synchronization*: If `try_lock_shared()` returns `true`, prior `unlock()` operations on the same object synchronize with ([intro.multithread]) this operation.

21 *Return type*: `bool`.

22 *Returns*: `true` if the shared lock was acquired, otherwise `false`.

23 *Throws*: Nothing.

32.6.4.4.2 Class `shared_mutex` [thread.sharedmutex.class]

```
namespace std {
  class shared_mutex {
  public:
    constexpr shared_mutex();
    constexpr ~shared_mutex();

    shared_mutex(const shared_mutex&) = delete;
    shared_mutex& operator=(const shared_mutex&) = delete;

    // exclusive ownership
    constexpr void lock();                  // blocking
    constexpr bool try_lock();
    constexpr void unlock();

    // shared ownership
    constexpr void lock_shared();           // blocking
    constexpr bool try_lock_shared();
    constexpr void unlock_shared();

    using native_handle_type = implementation-defined;          // see [thread.req.native]
    native_handle_type native_handle();                         // see [thread.req.native]
  };
}
```

1 The class `shared_mutex` provides a non-recursive mutex with shared ownership semantics.

2 The class `shared_mutex` meets all of the shared mutex requirements ([thread.sharedmutex.requirements]). It is a standard-layout class ([class.prop]).

3

The behavior of a program is undefined if

(3.1) — it destroys a `shared_mutex` object owned by any thread,

(3.2) — a thread attempts to recursively gain any ownership of a `shared_mutex`, or

(3.3) — a thread terminates while possessing any ownership of a `shared_mutex`.

4 `shared_mutex` may be a synonym for `shared_timed_mutex`.

32.6.4.5 Shared timed mutex types [thread.sharedtimedmutex.requirements]

32.6.4.5.1 General [thread.sharedtimedmutex.requirements.general]

1 The standard library type `shared_timed_mutex` is a *shared timed mutex type*. Shared timed mutex types meet the requirements of timed mutex types ([thread.timedmutex.requirements]), shared mutex types ([thread.sharedmutex.requirements]), and additionally meet the requirements set out below. In this description, `m` denotes an object of a shared timed mutex type, `rel_time` denotes an object of an instantiation of `duration` ([time.duration]), and `abs_time` denotes an object of an instantiation of `time_point`.

[*Note 1*: The shared timed mutex types meet the *Cpp17SharedTimedLockable* requirements ([thread.req.lockable.shared.timed]). — *end note*]

2 The expression `m.try_lock_shared_for(rel_time)` is well-formed and has the following semantics:

3 *Preconditions*: The calling thread has no ownership of the mutex.

4 *Effects*: Attempts to obtain shared lock ownership for the calling thread within the relative timeout ([thread.req.timing]) specified by `rel_time`. If the time specified by `rel_time` is less than or equal to `rel_time.zero()`, the function attempts to obtain ownership without blocking (as if by calling `try_lock_shared()`). The function returns within the timeout specified by `rel_time` only if it has obtained shared ownership of the mutex object.

[*Note 2*: As with `try_lock()`, there is no guarantee that ownership will be obtained if the lock is available, but implementations are expected to make a strong effort to do so. — *end note*]

If an exception is thrown then a shared lock has not been acquired for the current thread.

5 *Synchronization*: If `try_lock_shared_for()` returns `true`, prior `unlock()` operations on the same object synchronize with ([intro.multithread]) this operation.

6 *Return type*: `bool`.

7 *Returns*: `true` if the shared lock was acquired, otherwise `false`.

Throws: Timeout-related exceptions ([thread.req.timing]).

The expression `m.try_lock_shared_until(abs_time)` is well-formed and has the following semantics:

Preconditions: The calling thread has no ownership of the mutex.

Effects: The function attempts to obtain shared ownership of the mutex. If `abs_time` has already passed, the function attempts to obtain shared ownership without blocking (as if by calling `try_lock_shared()`). The function returns before the absolute timeout ([thread.req.timing]) specified by `abs_time` only if it has obtained shared ownership of the mutex object.

[*Note 3*: As with `try_lock()`, there is no guarantee that ownership will be obtained if the lock is available, but implementations are expected to make a strong effort to do so. — *end note*]

If an exception is thrown then a shared lock has not been acquired for the current thread.

Synchronization: If `try_lock_shared_until()` returns `true`, prior `unlock()` operations on the same object synchronize with ([intro.multithread]) this operation.

Return type: `bool`.

Returns: `true` if the shared lock was acquired, otherwise `false`.

Throws: Timeout-related exceptions ([thread.req.timing]).

32.6.4.5.2 Class `shared_timed_mutex` [thread.sharedtimedmutex.class]

```
namespace std {
  class shared_timed_mutex {
  public:
    constexpr shared_timed_mutex();
    constexpr ~shared_timed_mutex();

    shared_timed_mutex(const shared_timed_mutex&) = delete;
    shared_timed_mutex& operator=(const shared_timed_mutex&) = delet
e;

    // exclusive ownership
    constexpr void lock();                  // blocking
    constexpr bool try_lock();
    template<class Rep, class Period>
      constexpr bool try_lock_for(const chrono::duration<Rep, Period>
& rel_time);
    template<class Clock, class Duration>
```

```
    constexpr bool try_lock_until(const chrono::time_point<Clock, D
uration>& abs_time);
    constexpr void unlock();

    // shared ownership
    constexpr void lock_shared();          // blocking
    constexpr bool try_lock_shared();
    template<class Rep, class Period>
      constexpr bool try_lock_shared_for(const chrono::duration<Rep,
Period>& rel_time);
    template<class Clock, class Duration>
      constexpr bool try_lock_shared_until(const chrono::time_point<C
lock, Duration>& abs_time);
    constexpr void unlock_shared();
  };
}
```

1 The class `shared_timed_mutex` provides a non-recursive mutex with shared ownership semantics.

2 The class `shared_timed_mutex` meets all of the shared timed mutex requirements ([thread.sharedtimedmutex.requirements]). It is a standard-layout class ([class.prop]).

3 The behavior of a program is undefined if

(3.1) — it destroys a `shared_timed_mutex` object owned by any thread,

(3.2) — a thread attempts to recursively gain any ownership of a `shared_timed_mutex`, or

(3.3) — a thread terminates while possessing any ownership of a `shared_timed_mutex`.

32.6.5 Locks [thread.lock]

32.6.5.1 General [thread.lock.general]

1 A *lock* is an object that holds a reference to a lockable object and may unlock the lockable object during the lock's destruction (such as when leaving block scope). An execution agent may use a lock to aid in managing ownership of a lockable object in an exception safe manner. A lock is said to *own* a lockable object if it is currently managing the ownership of that lockable object for an execution agent. A lock does not manage the lifetime of the lockable object it references.

[*Note 1*: Locks are intended to ease the burden of unlocking the lockable object under both normal and exceptional circumstances. — *end note*]

2

Some lock constructors take tag types which describe what should be done with the lockable object during the lock's construction.

```
namespace std {
  struct defer_lock_t  { };      // do not acquire ownership of the mutex
  struct try_to_lock_t { };      // try to acquire ownership of the mutex
                                 // without blocking
  struct adopt_lock_t  { };      // assume the calling thread has already
                                 // obtained mutex ownership and manage it

  inline constexpr defer_lock_t   defer_lock { };
  inline constexpr try_to_lock_t  try_to_lock { };
  inline constexpr adopt_lock_t   adopt_lock { };
}
```

32.6.5.2 Class template `lock_guard` [thread.lock.guard]

```
namespace std {
  template<class Mutex>
  class lock_guard {
  public:
    using mutex_type = Mutex;

    constexpr explicit lock_guard(mutex_type& m);
    constexpr lock_guard(mutex_type& m, adopt_lock_t);
    constexpr ~lock_guard();

    lock_guard(const lock_guard&) = delete;
    lock_guard& operator=(const lock_guard&) = delete;

  private:
    mutex_type& pm;              // exposition only
  };
}
```

1 An object of type `lock_guard` controls the ownership of a lockable object within a scope. A `lock_guard` object maintains ownership of a lockable object throughout the `lock_guard` object's lifetime. The behavior of a program is undefined if the lockable object referenced by `pm` does not exist for the entire lifetime of the `lock_guard` object. The supplied `Mutex` type shall meet the *Cpp17BasicLockable* requirements ([thread.req.lockable.basic]).

```
constexpr explicit lock_guard(mutex_type& m);
```

2 *Effects*: Initializes `pm` with `m`. Calls `m.lock()`.

```
constexpr lock_guard(mutex_type& m, adopt_lock_t);
```

3 *Preconditions*: The calling thread holds a non-shared lock on `m`.

4 *Effects*: Initializes `pm` with `m`.

5 *Throws*: Nothing.

```
constexpr ~lock_guard();
```

6 *Effects*: Equivalent to: `pm.unlock()`

32.6.5.3 Class template `scoped_lock` [thread.lock.scoped]

```
namespace std {
  template<class... MutexTypes>
  class scoped_lock {
  public:
    using mutex_type = see below;      // Only if sizeof...(MutexTypes) ==
1 is true

    constexpr explicit scoped_lock(MutexTypes&... m);
    constexpr explicit scoped_lock(adopt_lock_t, MutexTypes&... m);
    constexpr ~scoped_lock();

    scoped_lock(const scoped_lock&) = delete;
    scoped_lock& operator=(const scoped_lock&) = delete;

  private:
    tuple<MutexTypes&...> pm;    // exposition only
  };
}
```

1 An object of type `scoped_lock` controls the ownership of lockable objects within a scope. A `scoped_lock` object maintains ownership of lockable objects throughout the `scoped_lock` object's lifetime. The behavior of a program is undefined if the lockable objects referenced by `pm` do not exist for the entire lifetime of the `scoped_lock` object.

(1.1) — If `sizeof...(MutexTypes)` is one, let `Mutex` denote the sole type constituting the pack `MutexTypes`. `Mutex` shall meet the *Cpp17BasicLockable* requirements ([thread.req.lockable.basic]). The member *typedef-name* `mutex_type` denotes the same type as `Mutex`.

(1.2) — Otherwise, all types in the template parameter pack `MutexTypes` shall meet the *Cpp17Lockable* requirements ([thread.req.lockable.req]) and there is no member `mutex_type`.

```
constexpr explicit scoped_lock(MutexTypes&... m);
```

2

Effects: Initializes `pm` with `tie(m...)`. Then if `sizeof...(MutexTypes)` is `0`, no effects. Otherwise if `sizeof...(MutexTypes)` is `1`, then `m.lock()`. Otherwise, `lock(m...)`.

```
constexpr explicit scoped_lock(adopt_lock_t, MutexTypes&... m);
```

3 *Preconditions*: The calling thread holds a non-shared lock on each element of `m`.

4 *Effects*: Initializes `pm` with `tie(m...)`.

5 *Throws*: Nothing.

```
constexpr ~scoped_lock();
```

6 *Effects*: For all `i` in `[0, sizeof...(MutexTypes))`, `get<i>(pm).unlock()`.

32.6.5.4 Class template `unique_lock` [thread.lock.unique]

32.6.5.4.1 General [thread.lock.unique.general]

```
namespace std {
  template<class Mutex>
  class unique_lock {
  public:
    using mutex_type = Mutex;

    // [thread.lock.unique.cons], construct/copy/destroy
    constexpr unique_lock() noexcept;
    constexpr explicit unique_lock(mutex_type& m);
    constexpr unique_lock(mutex_type& m, defer_lock_t) noexcept;
    constexpr unique_lock(mutex_type& m, try_to_lock_t);
    constexpr unique_lock(mutex_type& m, adopt_lock_t);
    template<class Clock, class Duration>
      constexpr unique_lock(mutex_type& m, const chrono::time_point<C
lock, Duration>& abs_time);
    template<class Rep, class Period>
      constexpr unique_lock(mutex_type& m, const chrono::duration<Re
p, Period>& rel_time);
    constexpr ~unique_lock();

    unique_lock(const unique_lock&) = delete;
    unique_lock& operator=(const unique_lock&) = delete;

    constexpr unique_lock(unique_lock&& u) noexcept;
    constexpr unique_lock& operator=(unique_lock&& u) noexcept;

    // [thread.lock.unique.locking], locking
```

```
    constexpr void lock();
    constexpr bool try_lock();

    template<class Rep, class Period>
      constexpr bool try_lock_for(const chrono::duration<Rep, Period>
& rel_time);
    template<class Clock, class Duration>
      constexpr bool try_lock_until(const chrono::time_point<Clock, D
uration>& abs_time);

    constexpr void unlock();

    // [thread.lock.unique.mod], modifiers
    constexpr void swap(unique_lock& u) noexcept;
    constexpr mutex_type* release() noexcept;

    // [thread.lock.unique.obs], observers
    constexpr bool owns_lock() const noexcept;
    constexpr explicit operator bool() const noexcept;
    constexpr mutex_type* mutex() const noexcept;

  private:
    mutex_type* pm;               // exposition only
    bool owns;                    // exposition only
  };
}
```

1 An object of type `unique_lock` controls the ownership of a lockable object within a scope. Ownership of the lockable object may be acquired at construction or after construction, and may be transferred, after acquisition, to another `unique_lock` object. Objects of type `unique_lock` are not copyable but are movable. The behavior of a program is undefined if the contained pointer `pm` is not null and the lockable object pointed to by `pm` does not exist for the entire remaining lifetime ([basic.life]) of the `unique_lock` object. The supplied `Mutex` type shall meet the *Cpp17BasicLockable* requirements ([thread.req.lockable.basic]).

32.6.5.4.2 Constructors, destructor, and assignment [thread.lock.unique.cons]

```
constexpr unique_lock() noexcept;
```

1 *Postconditions*: `pm == nullptr` and `owns == false`.

```
constexpr explicit unique_lock(mutex_type& m);
```

2 *Effects*: Calls `m.lock()`.

3 *Postconditions*: `pm == addressof(m)` and `owns == true`.

```
constexpr unique_lock(mutex_type& m, defer_lock_t) noexcept;
```

4 *Postconditions*: pm == addressof(m) and owns == false.

```
constexpr unique_lock(mutex_type& m, try_to_lock_t);
```

5 *Preconditions*: The supplied Mutex type meets the *Cpp17Lockable* requirements ([thread.req.lockable.req]).

6 *Effects*: Calls m.try_lock().

7 *Postconditions*: pm == addressof(m) and owns == res, where res is the value returned by the call to m.try_lock().

```
constexpr unique_lock(mutex_type& m, adopt_lock_t);
```

8 *Preconditions*: The calling thread holds a non-shared lock on m.

9 *Postconditions*: pm == addressof(m) and owns == true.

10 *Throws*: Nothing.

```
template<class Clock, class Duration>
  constexpr unique_lock(mutex_type& m, const chrono::time_point<Clock,
Duration>& abs_time);
```

11 *Preconditions*: The supplied Mutex type meets the *Cpp17TimedLockable* requirements ([thread.req.lockable.timed]).

12 *Effects*: Calls m.try_lock_until(abs_time).

13 *Postconditions*: pm == addressof(m) and owns == res, where res is the value returned by the call to m.try_lock_until(abs_time).

```
template<class Rep, class Period>
  constexpr unique_lock(mutex_type& m, const chrono::duration<Rep,
Period>& rel_time);
```

14 *Preconditions*: The supplied Mutex type meets the *Cpp17TimedLockable* requirements ([thread.req.lockable.timed]).

15 *Effects*: Calls m.try_lock_for(rel_time).

16 *Postconditions*: pm == addressof(m) and owns == res, where res is the value returned by the call to m.try_lock_for(rel_time).

```
constexpr unique_lock(unique_lock&& u) noexcept;
```

17 *Postconditions*: pm == u_p.pm and owns == u_p.owns (where u_p is the state of u just prior to this construction), u.pm == 0 and u.owns == false.

```
constexpr unique_lock& operator=(unique_lock&& u) noexcept;
```

18 *Effects*: Equivalent to: unique_lock(std::move(u)).swap(*this)

19 *Returns*: `*this`.

```
constexpr ~unique_lock();
```

20 *Effects*: If `owns` calls `pm->unlock()`.

32.6.5.4.3 Locking [thread.lock.unique.locking]

```
constexpr void lock();
```

1 *Effects*: As if by `pm->lock()`.

2 *Postconditions*: `owns == true`.

3 *Throws*: Any exception thrown by `pm->lock()`. `system_error` when an exception is required ([thread.req.exception]).

4 *Error conditions*:

(4.1) — `operation_not_permitted` — if `pm` is `nullptr`.

(4.2) — `resource_deadlock_would_occur` — if on entry `owns` is `true`.

```
constexpr bool try_lock();
```

5 *Preconditions*: The supplied `Mutex` meets the *Cpp17Lockable* requirements ([thread.req.lockable.req]).

6 *Effects*: As if by `pm->try_lock()`.

7 *Postconditions*: `owns == res`, where `res` is the value returned by `pm->try_lock()`.

8 *Returns*: The value returned by `pm->try_lock()`.

9 *Throws*: Any exception thrown by `pm->try_lock()`. `system_error` when an exception is required ([thread.req.exception]).

10 *Error conditions*:

(10.1) — `operation_not_permitted` — if `pm` is `nullptr`.

(10.2) — `resource_deadlock_would_occur` — if on entry `owns` is `true`.

```
template<class Clock, class Duration>
  constexpr bool try_lock_until(const chrono::time_point<Clock,
Duration>& abs_time);
```

11 *Preconditions*: The supplied `Mutex` type meets the *Cpp17TimedLockable* requirements ([thread.req.lockable.timed]).

12 *Effects*: As if by `pm->try_lock_until(abs_time)`.

13 *Postconditions*: `owns == res`, where `res` is the value returned by `pm->try_lock_until(abs_time)`.

14 *Returns*: The value returned by `pm->try_lock_until(abs_time)`.

15 *Throws*: Any exception thrown by `pm->try_lock_until(abstime)`. `system_error` when an exception is required ([thread.req.exception]).

16 *Error conditions*:

(16.1) — `operation_not_permitted` — if `pm` is `nullptr`.

(16.2) — `resource_deadlock_would_occur` — if on entry `owns` is `true`.

```
template<class Rep, class Period>
  constexpr bool try_lock_for(const chrono::duration<Rep, Period>&
rel_time);
```

17 *Preconditions*: The supplied `Mutex` type meets the *Cpp17TimedLockable* requirements ([thread.req.lockable.timed]).

18 *Effects*: As if by `pm->try_lock_for(rel_time)`.

19 *Postconditions*: `owns == res`, where `res` is the value returned by `pm->try_lock_for(rel_time)`.

20 *Returns*: The value returned by `pm->try_lock_for(rel_time)`.

21 *Throws*: Any exception thrown by `pm->try_lock_for(rel_time)`. `system_error` when an exception is required ([thread.req.exception]).

22 *Error conditions*:

(22.1) — `operation_not_permitted` — if `pm` is `nullptr`.

(22.2) — `resource_deadlock_would_occur` — if on entry `owns` is `true`.

```
constexpr void unlock();
```

23 *Effects*: As if by `pm->unlock()`.

24 *Postconditions*: `owns == false`.

25 *Throws*: `system_error` when an exception is required ([thread.req.exception]).

26 *Error conditions*:

(26.1) — `operation_not_permitted` — if on entry `owns` is `false`.

32.6.5.4.4 Modifiers [thread.lock.unique.mod]

```
constexpr void swap(unique_lock& u) noexcept;
```

1 *Effects*: Swaps the data members of `*this` and `u`.

```
constexpr mutex_type* release() noexcept;
```

2 *Postconditions*: `pm == 0` and `owns == false`.

3 *Returns*: The previous value of pm.

```
template<class Mutex>
  constexpr void swap(unique_lock<Mutex>& x, unique_lock<Mutex>& y)
noexcept;
```

4 *Effects*: As if by x.swap(y).

32.6.5.4.5 Observers [thread.lock.unique.obs]

```
constexpr bool owns_lock() const noexcept;
```

1 *Returns*: owns.

```
constexpr explicit operator bool() const noexcept;
```

2 *Returns*: owns.

```
constexpr mutex_type *mutex() const noexcept;
```

3 *Returns*: pm.

32.6.5.5 Class template shared_lock [thread.lock.shared]

32.6.5.5.1 General [thread.lock.shared.general]

```
namespace std {
  template<class Mutex>
  class shared_lock {
  public:
    using mutex_type = Mutex;

    // [thread.lock.shared.cons], construct/copy/destroy
    constexpr shared_lock() noexcept;
    constexpr explicit shared_lock(mutex_type& m);          // blocking
    constexpr shared_lock(mutex_type& m, defer_lock_t) noexcept;
    constexpr shared_lock(mutex_type& m, try_to_lock_t);
    constexpr shared_lock(mutex_type& m, adopt_lock_t);
    template<class Clock, class Duration>
      constexpr shared_lock(mutex_type& m, const chrono::time_point<C
lock, Duration>& abs_time);
    template<class Rep, class Period>
      constexpr shared_lock(mutex_type& m, const chrono::duration<Re
p, Period>& rel_time);
    constexpr ~shared_lock();

    shared_lock(const shared_lock&) = delete;
    shared_lock& operator=(const shared_lock&) = delete;
```

```
    constexpr shared_lock(shared_lock&& u) noexcept;
    constexpr shared_lock& operator=(shared_lock&& u) noexcept;

    // [thread.lock.shared.locking], locking
    constexpr void lock();                                  // blocking
    constexpr bool try_lock();
    template<class Rep, class Period>
      constexpr bool try_lock_for(const chrono::duration<Rep, Period>
& rel_time);
    template<class Clock, class Duration>
      constexpr bool try_lock_until(const chrono::time_point<Clock, D
uration>& abs_time);
    constexpr void unlock();

    // [thread.lock.shared.mod], modifiers
    constexpr void swap(shared_lock& u) noexcept;
    constexpr mutex_type* release() noexcept;

    // [thread.lock.shared.obs], observers
    constexpr bool owns_lock() const noexcept;
    constexpr explicit operator bool() const noexcept;
    constexpr mutex_type* mutex() const noexcept;

  private:
    mutex_type* pm;                                 // exposition only
    bool owns;                                      // exposition only
  };
}
```

1 An object of type `shared_lock` controls the shared ownership of a lockable object within a scope. Shared ownership of the lockable object may be acquired at construction or after construction, and may be transferred, after acquisition, to another `shared_lock` object. Objects of type `shared_lock` are not copyable but are movable. The behavior of a program is undefined if the contained pointer `pm` is not null and the lockable object pointed to by `pm` does not exist for the entire remaining lifetime ([basic.life]) of the `shared_lock` object. The supplied `Mutex` type shall meet the *Cpp17SharedLockable* requirements ([thread.req.lockable.shared]).

32.6.5.5.2 Constructors, destructor, and assignment [thread.lock.shared.cons]

```
constexpr shared_lock() noexcept;
```

1 *Postconditions*: `pm == nullptr` and `owns == false`.

```
constexpr explicit shared_lock(mutex_type& m);
```

2 *Effects*: Calls `m.lock_shared()`.

Postconditions: `pm == addressof(m)` and `owns == true`.

```
constexpr shared_lock(mutex_type& m, defer_lock_t) noexcept;
```

Postconditions: `pm == addressof(m)` and `owns == false`.

```
constexpr shared_lock(mutex_type& m, try_to_lock_t);
```

Effects: Calls `m.try_lock_shared()`.

Postconditions: `pm == addressof(m)` and `owns == res` where `res` is the value returned by the call to `m.try_lock_shared()`.

```
constexpr shared_lock(mutex_type& m, adopt_lock_t);
```

Preconditions: The calling thread holds a shared lock on `m`.

Postconditions: `pm == addressof(m)` and `owns == true`.

```
template<class Clock, class Duration>
  constexpr shared_lock(mutex_type& m,
                        const chrono::time_point<Clock, Duration>& abs_time);
```

Preconditions: `Mutex` meets the *Cpp17SharedTimedLockable* requirements ([thread.req.lockable.shared.timed]).

Effects: Calls `m.try_lock_shared_until(abs_time)`.

Postconditions: `pm == addressof(m)` and `owns == res` where `res` is the value returned by the call to `m.try_lock_shared_until(abs_time)`.

```
template<class Rep, class Period>
  constexpr shared_lock(mutex_type& m,
                        const chrono::duration<Rep, Period>& rel_time);
```

Preconditions: `Mutex` meets the *Cpp17SharedTimedLockable* requirements ([thread.req.lockable.shared.timed]).

Effects: Calls `m.try_lock_shared_for(rel_time)`.

Postconditions: `pm == addressof(m)` and `owns == res` where `res` is the value returned by the call to `m.try_lock_shared_for(rel_time)`.

```
constexpr ~shared_lock();
```

Effects: If `owns` calls `pm->unlock_shared()`.

```
constexpr shared_lock(shared_lock&& sl) noexcept;
```

Postconditions: `pm == sl_p.pm` and `owns == sl_p.owns` (where `sl_p` is the state of `sl` just prior to this construction), `sl.pm == nullptr` and `sl.owns == false`.

```
constexpr shared_lock& operator=(shared_lock&& sl) noexcept;
```

17 *Effects*: Equivalent to: `shared_lock(std::move(sl)).swap(*this)`

18 *Returns*: `*this`.

32.6.5.5.3 Locking [thread.lock.shared.locking]

```
constexpr void lock();
```

1 *Effects*: As if by `pm->lock_shared()`.

2 *Postconditions*: `owns == true`.

3 *Throws*: Any exception thrown by `pm->lock_shared()`. `system_error` when an exception is required ([thread.req.exception]).

4 *Error conditions*:

(4.1) — `operation_not_permitted` — if `pm` is `nullptr`.

(4.2) — `resource_deadlock_would_occur` — if on entry `owns` is `true`.

```
constexpr bool try_lock();
```

5 *Effects*: As if by `pm->try_lock_shared()`.

6 *Postconditions*: `owns == res`, where `res` is the value returned by the call to `pm->try_lock_shared()`.

7 *Returns*: The value returned by the call to `pm->try_lock_shared()`.

8 *Throws*: Any exception thrown by `pm->try_lock_shared()`. `system_error` when an exception is required ([thread.req.exception]).

9 *Error conditions*:

(9.1) — `operation_not_permitted` — if `pm` is `nullptr`.

(9.2) — `resource_deadlock_would_occur` — if on entry `owns` is `true`.

```
template<class Clock, class Duration>
  constexpr bool try_lock_until(const chrono::time_point<Clock,
Duration>& abs_time);
```

10 *Preconditions*: `Mutex` meets the *Cpp17SharedTimedLockable* requirements ([thread.req.lockable.shared.timed]).

11 *Effects*: As if by `pm->try_lock_shared_until(abs_time)`.

12 *Postconditions*: `owns == res`, where `res` is the value returned by the call to `pm->try_lock_shared_until(abs_time)`.

13 *Returns*: The value returned by the call to `pm->try_lock_shared_until(abs_time)`.

14

Throws: Any exception thrown by `pm->try_lock_shared_until(abs_time)`. `system_error` when an exception is required ([thread.req.exception]).

15 *Error conditions*:

(15.1) — `operation_not_permitted` — if `pm` is `nullptr`.

(15.2) — `resource_deadlock_would_occur` — if on entry `owns` is `true`.

```
template<class Rep, class Period>
  constexpr bool try_lock_for(const chrono::duration<Rep, Period>&
rel_time);
```

16 *Preconditions*: `Mutex` meets the *Cpp17SharedTimedLockable* requirements ([thread.req.lockable.shared.timed]).

17 *Effects*: As if by `pm->try_lock_shared_for(rel_time)`.

18 *Postconditions*: `owns == res`, where `res` is the value returned by the call to `pm->try_lock_shared_for(rel_time)`.

19 *Returns*: The value returned by the call to `pm->try_lock_shared_for(rel_time)`.

20 *Throws*: Any exception thrown by `pm->try_lock_shared_for(rel_time)`. `system_error` when an exception is required ([thread.req.exception]).

21 *Error conditions*:

(21.1) — `operation_not_permitted` — if `pm` is `nullptr`.

(21.2) — `resource_deadlock_would_occur` — if on entry `owns` is `true`.

```
constexpr void unlock();
```

22 *Effects*: As if by `pm->unlock_shared()`.

23 *Postconditions*: `owns == false`.

24 *Throws*: `system_error` when an exception is required ([thread.req.exception]).

25 *Error conditions*:

(25.1) — `operation_not_permitted` — if on entry `owns` is `false`.

32.6.5.5.4 Modifiers [thread.lock.shared.mod]

```
constexpr void swap(shared_lock& sl) noexcept;
```

1 *Effects*: Swaps the data members of `*this` and `sl`.

```
constexpr mutex_type* release() noexcept;
```

2 *Postconditions*: `pm == nullptr` and `owns == false`.

3 *Returns*: The previous value of pm.

```
template<class Mutex>
  constexpr void swap(shared_lock<Mutex>& x, shared_lock<Mutex>& y)
noexcept;
```

4 *Effects*: As if by x.swap(y).

32.6.5.5.5 Observers [thread.lock.shared.obs]

```
constexpr bool owns_lock() const noexcept;
```

1 *Returns*: owns.

```
constexpr explicit operator bool() const noexcept;
```

2 *Returns*: owns.

```
constexpr mutex_type* mutex() const noexcept;
```

3 *Returns*: pm.

32.6.6 Generic locking algorithms [thread.lock.algorithm]

```
template<class L1, class L2, class... L3> constexpr int try_lock(L1&,
L2&, L3&...);
```

1 *Preconditions*: Each template parameter type meets the *Cpp17Lockable* requirements.

> [*Note 1*: The unique_lock class template meets these requirements when suitably instantiated. — *end note*]

2 *Effects*: Calls try_lock() for each argument in order beginning with the first until all arguments have been processed or a call to try_lock() fails, either by returning false or by throwing an exception. If a call to try_lock() fails, unlock() is called for all prior arguments with no further calls to try_lock().

3 *Returns*: -1 if all calls to try_lock() returned true, otherwise a zero-based index value that indicates the argument for which try_lock() returned false.

```
template<class L1, class L2, class... L3> constexpr void lock(L1&, L2&,
L3&...);
```

4 *Preconditions*: Each template parameter type meets the *Cpp17Lockable* requirements.

[*Note 2*: The `unique_lock` class template meets these requirements when suitably instantiated. — *end note*]

5 *Effects*: All arguments are locked via a sequence of calls to `lock()`, `try_lock()`, or `unlock()` on each argument. The sequence of calls does not result in deadlock, but is otherwise unspecified.

[*Note 3*: A deadlock avoidance algorithm such as try-and-back-off can be used, but the specific algorithm is not specified to avoid over-constraining implementations. — *end note*]

If a call to `lock()` or `try_lock()` throws an exception, `unlock()` is called for any argument that had been locked by a call to `lock()` or `try_lock()`.

32.6.7 Call once [thread.once]

32.6.7.1 Struct `once_flag` [thread.once.onceflag]

```
namespace std {
  struct once_flag {
    constexpr once_flag() noexcept;

    once_flag(const once_flag&) = delete;
    once_flag& operator=(const once_flag&) = delete;
  };
}
```

1 The class `once_flag` is an opaque data structure that `call_once` uses to initialize data without causing a data race or deadlock.

```
constexpr once_flag() noexcept;
```

2 *Synchronization*: The construction of a `once_flag` object is not synchronized.

3 *Postconditions*: The object's internal state is set to indicate to an invocation of `call_once` with the object as its initial argument that no function has been called.

32.6.7.2 Function `call_once` [thread.once.callonce]

```
template<class Callable, class... Args>
  constexpr void call_once(once_flag& flag, Callable&& func, Args&&...
args);
```

1 *Mandates*: `is_invocable_v<Callable, Args...>` is `true`.

2 *Effects*: An execution of `call_once` that does not call its `func` is a *passive* execution. An execution of `call_once` that calls its `func` is an *active* execution.

An active execution evaluates *INVOKE*(`std::forward<Callable>(func)`, `std::forward<Args>(args)...`) ([func.require]). If such a call to `func` throws an exception the execution is *exceptional*, otherwise it is *returning*. An exceptional execution propagates the exception to the caller of `call_once`. Among all executions of `call_once` for any given `once_flag`: at most one is a returning execution; if there is a returning execution, it is the last active execution; and there are passive executions only if there is a returning execution.

[*Note 1*: Passive executions allow other threads to reliably observe the results produced by the earlier returning execution. — *end note*]

3 *Synchronization*: For any given `once_flag`: all active executions occur in a total order; completion of an active execution synchronizes with the start of the next one in this total order; and the returning execution synchronizes with the return from all passive executions.

4 *Throws*: `system_error` when an exception is required ([thread.req.exception]), or any exception thrown by `func`.

Timing specifications

32.2.4 Timing specifications [thread.req.timing]

1 Several functions described in this Clause take an argument to specify a timeout. These timeouts are specified as either a `duration` or a `time_point` type as specified in [time].

2 Implementations necessarily have some delay in returning from a timeout. Any overhead in interrupt response, function return, and scheduling induces a “quality of implementation” delay, expressed as duration D_i. Ideally, this delay would be zero. Further, any contention for processor and memory resources induces a “quality of management” delay, expressed as duration D_m. The delay durations may vary from timeout to timeout, but in all cases shorter is better.

3 The functions whose names end in `_for` take an argument that specifies a duration. These functions produce relative timeouts. Implementations should use a steady clock to measure time for these functions. Given a duration argument D_t, the real-time duration of the timeout is $D_t + D_i + D_m$.

4 The functions whose names end in `_until` take an argument that specifies a time point. These functions produce absolute timeouts. Implementations should use the clock specified in the time point to measure time for these functions. Given a clock time point argument C_t, the clock time point of the return from timeout should be $C_t + D_i + D_m$ when the clock is not adjusted during the timeout. If the clock is adjusted

to the time C_a during the timeout, the behavior should be as follows:

(4.1) — If $C_a > C_t$, the waiting function should wake as soon as possible, i.e., $C_a + D_i + D_m$, since the timeout is already satisfied. This specification may result in the total duration of the wait decreasing when measured against a steady clock.

(4.2) — If $C_a \leq C_t$, the waiting function should not time out until `Clock::now()` returns a time $C_n \geq C_t$, i.e., waking at $C_t + D_i + D_m$.

[*Note 1*: When the clock is adjusted backwards, this specification can result in the total duration of the wait increasing when measured against a steady clock. When the clock is adjusted forwards, this specification can result in the total duration of the wait decreasing when measured against a steady clock. — *end note*]

An implementation returns from such a timeout at any point from the time specified above to the time it would return from a steady-clock relative timeout on the difference between C_t and the time point of the call to the `_until` function.

Recommended practice: Implementations should decrease the duration of the wait when the clock is adjusted forwards.

6 The resolution of timing provided by an implementation depends on both operating system and hardware. The finest resolution provided by an implementation is called the *native resolution*.

7 Implementation-provided clocks that are used for these functions meet the *Cpp17TrivialClock* requirements ([time.clock.req]).

8 A function that takes an argument which specifies a timeout will throw if, during its execution, a clock, time point, or time duration throws an exception. Such exceptions are referred to as *timeout-related exceptions*.

[*Note 3*: Instantiations of clock, time point and duration types supplied by the implementation as specified in [time.clock] do not throw exceptions. — *end note*]

? During constant evaluation [expr.const] all functions taking timeout argument will return immediately as if the time requested already passed.

Condition variables

32.7 Condition variables [thread.condition]

32.7.1 General [thread.condition.general]

1 Condition variables provide synchronization primitives used to block a thread until notified by some other thread that some condition is met or until a system time is reached. Class `condition_variable` provides a condition variable that can only wait on an object of type `unique_lock<mutex>`, allowing the implementation to be

more efficient. Class `condition_variable_any` provides a general condition variable that can wait on objects of user-supplied lock types.

2 Condition variables permit concurrent invocation of the `wait`, `wait_for`, `wait_until`, `notify_one` and `notify_all` member functions.

3 The executions of `notify_one` and `notify_all` are atomic. The executions of `wait`, `wait_for`, and `wait_until` are performed in three atomic parts:

1. the release of the mutex and entry into the waiting state;
2. the unblocking of the wait; and
3. the reacquisition of the lock.

4 The implementation behaves as if all executions of `notify_one`, `notify_all`, and each part of the `wait`, `wait_for`, and `wait_until` executions are executed in a single unspecified total order consistent with the "happens before" order.

5 Condition variable construction and destruction need not be synchronized.

32.7.2 Header <condition_variable> synopsis [condition.variable.syn]

```
namespace std {
  // [thread.condition.condvar], class condition_variable
  class condition_variable;
  // [thread.condition.condvarany], class condition_variable_any
  class condition_variable_any;

  // [thread.condition.nonmember], non-member functions
  constexpr void notify_all_at_thread_exit(condition_variable& cond,
unique_lock<mutex> lk);

  enum class cv_status { no_timeout, timeout };
}
```

32.7.3 Non-member functions [thread.condition.nonmember]

```
constexpr void notify_all_at_thread_exit(condition_variable& cond,
unique_lock<mutex> lk);
```

1 *Preconditions*: `lk` is locked by the calling thread and either

(1.1) — no other thread is waiting on `cond`, or

(1.2) — `lk.mutex()` returns the same value for each of the lock arguments supplied by all concurrently waiting (via `wait`, `wait_for`, or `wait_until`) threads.

2

Effects: Transfers ownership of the lock associated with `lk` into internal storage and schedules `cond` to be notified when the current thread exits, after all objects with thread storage duration associated with the current thread have been destroyed. This notification is equivalent to:

```
lk.unlock();
cond.notify_all();
```

3 *Synchronization*: The implied `lk.unlock()` call is sequenced after the destruction of all objects with thread storage duration associated with the current thread.

[*Note ?*: There is only one thread during constant evaluation and using this in such environment is a no-op. — *end note*]

32.7.4 Class `condition_variable` [thread.condition.condvar]

```
namespace std {
  class condition_variable {
  public:
    constexpr condition_variable();
    constexpr ~condition_variable();

    condition_variable(const condition_variable&) = delete;
    condition_variable& operator=(const condition_variable&) = delet
e;

    constexpr void notify_one() noexcept;
    constexpr void notify_all() noexcept;
    constexpr void wait(unique_lock<mutex>& lock);
    template<class Predicate>
      constexpr void wait(unique_lock<mutex>& lock, Predicate pred);
    template<class Clock, class Duration>
      constexpr cv_status wait_until(unique_lock<mutex>& lock,
                                     const chrono::time_point<Clock, Duration>
& abs_time);
    template<class Clock, class Duration, class Predicate>
      constexpr bool wait_until(unique_lock<mutex>& lock,
                                const chrono::time_point<Clock, Duration>& abs_
time,
                                Predicate pred);
    template<class Rep, class Period>
      constexpr cv_status wait_for(unique_lock<mutex>& lock,
                                   const chrono::duration<Rep, Period>& rel_tim
e);
    template<class Rep, class Period, class Predicate>
      constexpr bool wait_for(unique_lock<mutex>& lock,
```

```
                    const chrono::duration<Rep, Period>& rel_time,
                    Predicate pred);

    using native_handle_type = implementation-defined;          // see [t
hread.req.native]
    native_handle_type native_handle();                          // see
[thread.req.native]
  };
}
```

1 The class `condition_variable` is a standard-layout class ([class.prop]).

`constexpr condition_variable();`

2 *Throws*: `system_error` when an exception is required ([thread.req.-exception]).

3 *Error conditions*:

(3.1) — `resource_unavailable_try_again` — if some non-memory resource limitation prevents initialization.

`constexpr ~condition_variable();`

4 *Preconditions*: There is no thread blocked on `*this`.

[*Note 1*: That is, all threads have been notified; they can subsequently block on the lock specified in the wait. This relaxes the usual rules, which would have required all wait calls to happen before destruction. Only the notification to unblock the wait needs to happen before destruction. Undefined behavior ensues if a thread waits on `*this` once the destructor has been started, especially when the waiting threads are calling the wait functions in a loop or using the overloads of `wait`, `wait_for`, or `wait_until` that take a predicate. — *end note*]

`constexpr void notify_one() noexcept;`

5 *Effects*: If any threads are blocked waiting for `*this`, unblocks one of those threads.

`constexpr void notify_all() noexcept;`

6 *Effects*: Unblocks all threads that are blocked waiting for `*this`.

`constexpr void wait(unique_lock<mutex>& lock);`

7 *Preconditions*: `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread, and either

(7.1) — no other thread is waiting on this `condition_variable` object or

(7.2) — `lock.mutex()` returns the same value for each of the `lock` arguments supplied by all concurrently waiting (via `wait`, `wait_for`, or `wait_un-`

til) threads.

8 *Effects*:

(8.1) — Atomically calls `lock.unlock()` and blocks on `*this`.

(8.2) — When unblocked, calls `lock.lock()` (possibly blocking on the lock), then returns.

(8.3) — The function will unblock when signaled by a call to `notify_one()` or a call to `notify_all()`, or spuriously.

9 *Postconditions*: `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread.

10 *Throws*: Nothing.

11 *Remarks*: If the function fails to meet the postcondition, `terminate()` is invoked ([except.terminate]).

[*Note 2*: This can happen if the re-locking of the mutex throws an exception. — *end note*]

```
template<class Predicate>
  constexpr void wait(unique_lock<mutex>& lock, Predicate pred);
```

12 *Preconditions*: `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread, and either

(12.1) — no other thread is waiting on this `condition_variable` object or

(12.2) — `lock.mutex()` returns the same value for each of the `lock` arguments supplied by all concurrently waiting (via `wait`, `wait_for`, or `wait_until`) threads.

13 *Effects*: Equivalent to:

```
while (!pred())
  wait(lock);
```

14 *Postconditions*: `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread.

15 *Throws*: Any exception thrown by `pred`.

16 *Remarks*: If the function fails to meet the postcondition, `terminate()` is invoked ([except.terminate]).

[*Note 3*: This can happen if the re-locking of the mutex throws an exception. — *end note*]

```
template<class Clock, class Duration>
  constexpr cv_status wait_until(unique_lock<mutex>& lock,
                       const chrono::time_point<Clock, Duration>&
abs_time);
```

17 *Preconditions*: `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread, and either

(17.1) — no other thread is waiting on this `condition_variable` object or

(17.2) — `lock.mutex()` returns the same value for each of the `lock` arguments supplied by all concurrently waiting (via `wait`, `wait_for`, or `wait_until`) threads.

18 *Effects*:

(18.1) — Atomically calls `lock.unlock()` and blocks on `*this`.

(18.2) — When unblocked, calls `lock.lock()` (possibly blocking on the lock), then returns.

(18.3) — The function will unblock when signaled by a call to `notify_one()`, a call to `notify_all()`, expiration of the absolute timeout ([thread.req.timing]) specified by `abs_time`, or spuriously.

(18.4) — If the function exits via an exception, `lock.lock()` is called prior to exiting the function.

19 *Postconditions*: `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread.

20 *Returns*: `cv_status::timeout` if the absolute timeout ([thread.req.timing]) specified by `abs_time` expired, otherwise `cv_status::no_timeout`.

21 *Throws*: Timeout-related exceptions ([thread.req.timing]).

22 *Remarks*: If the function fails to meet the postcondition, `terminate()` is invoked ([except.terminate]).

[*Note 4*: This can happen if the re-locking of the mutex throws an exception. — *end note*]

```
template<class Rep, class Period>
  constexpr cv_status wait_for(unique_lock<mutex>& lock,
                               const chrono::duration<Rep, Period>& rel_time);
```

23 *Preconditions*: `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread, and either

(23.1) — no other thread is waiting on this `condition_variable` object or

(23.2) — `lock.mutex()` returns the same value for each of the `lock` arguments supplied by all concurrently waiting (via `wait`, `wait_for`, or `wait_until`) threads.

24 *Effects*: Equivalent to:

```
return wait_until(lock, chrono::steady_clock::now() + rel_time);
```

25 *Postconditions*: `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread.

26 *Returns*: `cv_status::timeout` if the relative timeout ([thread.req.timing]) specified by `rel_time` expired, otherwise `cv_status::no_timeout`.

27 *Throws*: Timeout-related exceptions ([thread.req.timing]).

28 *Remarks*: If the function fails to meet the postcondition, `terminate` is invoked ([except.terminate]).

[*Note 5*: This can happen if the re-locking of the mutex throws an exception. — *end note*]

```
template<class Clock, class Duration, class Predicate>
  constexpr bool wait_until(unique_lock<mutex>& lock,
                const chrono::time_point<Clock, Duration>& abs_time,
                Predicate pred);
```

29 *Preconditions*: `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread, and either

(29.1) — no other thread is waiting on this `condition_variable` object or

(29.2) — `lock.mutex()` returns the same value for each of the `lock` arguments supplied by all concurrently waiting (via `wait`, `wait_for`, or `wait_until`) threads.

30 *Effects*: Equivalent to:

```
while (!pred())
  if (wait_until(lock, abs_time) == cv_status::timeout)
    return pred();
return true;
```

31 *Postconditions*: `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread.

33 *Throws*: Timeout-related exceptions ([thread.req.timing]) or any exception thrown by `pred`.

34 *Remarks*: If the function fails to meet the postcondition, `terminate()` is invoked ([except.terminate]).

[*Note 7*: This can happen if the re-locking of the mutex throws an exception. — *end note*]

```
template<class Rep, class Period, class Predicate>
  constexpr bool wait_for(unique_lock<mutex>& lock,
                const chrono::duration<Rep, Period>& rel_time,
                Predicate pred);
```

35

Preconditions: `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread, and either

(35.1) — no other thread is waiting on this `condition_variable` object or

(35.2) — `lock.mutex()` returns the same value for each of the `lock` arguments supplied by all concurrently waiting (via `wait`, `wait_for`, or `wait_until`) threads.

36 *Effects*: Equivalent to:

```
return wait_until(lock, chrono::steady_clock::now() + rel_time, std::move(pred));
```

38 *Postconditions*: `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread.

40 *Throws*: Timeout-related exceptions ([thread.req.timing]) or any exception thrown by `pred`.

41 *Remarks*: If the function fails to meet the postcondition, `terminate()` is invoked ([except.terminate]).

[*Note 10*: This can happen if the re-locking of the mutex throws an exception. — *end note*]

32.7.5 Class `condition_variable_any` [thread.condition.condvarany]

32.7.5.1 General [thread.condition.condvarany.general]

1 In [thread.condition.condvarany], template arguments for template parameters named `Lock` shall meet the *Cpp17BasicLockable* requirements ([thread.req.lockable.basic]).

[*Note 1*: All of the standard mutex types meet this requirement. If a type other than one of the standard mutex types or a `unique_lock` wrapper for a standard mutex type is used with `condition_variable_any`, any necessary synchronization is assumed to be in place with respect to the predicate associated with the `condition_variable_any` instance. — *end note*]

```
namespace std {
  class condition_variable_any {
  public:
    constexpr condition_variable_any();
    constexpr ~condition_variable_any();

    condition_variable_any(const condition_variable_any&) = delete;
    condition_variable_any& operator=(const condition_variable_any
```

```
  &) = delete;

    constexpr void notify_one() noexcept;
    constexpr void notify_all() noexcept;

    // [thread.condvarany.wait], noninterruptible waits
    template<class Lock>
      constexpr void wait(Lock& lock);
    template<class Lock, class Predicate>
      constexpr void wait(Lock& lock, Predicate pred);

    template<class Lock, class Clock, class Duration>
      constexpr cv_status wait_until(Lock& lock, const chrono::tim
  e_point<Clock, Duration>& abs_time);
    template<class Lock, class Clock, class Duration, class Predic
  ate>
      constexpr bool wait_until(Lock& lock, const chrono::time_poi
  nt<Clock, Duration>& abs_time,
                        Predicate pred);
    template<class Lock, class Rep, class Period>
      constexpr cv_status wait_for(Lock& lock, const chrono::durat
  ion<Rep, Period>& rel_time);
    template<class Lock, class Rep, class Period, class Predicate>
      constexpr bool wait_for(Lock& lock, const chrono::duration<R
  ep, Period>& rel_time, Predicate pred);

    // [thread.condvarany.intwait], interruptible waits
    template<class Lock, class Predicate>
      constexpr bool wait(Lock& lock, stop_token stoken, Predicate
  pred);
    template<class Lock, class Clock, class Duration, class Predic
  ate>
      constexpr bool wait_until(Lock& lock, stop_token stoken,
                        const chrono::time_point<Clock, Duration>&
  abs_time, Predicate pred);
    template<class Lock, class Rep, class Period, class Predicate>
      constexpr bool wait_for(Lock& lock, stop_token stoken,
                      const chrono::duration<Rep, Period>& rel_tim
  e, Predicate pred);
  };
}

constexpr condition_variable_any();
```

2 *Throws*: bad_alloc or system_error when an exception is required ([thread.req.exception]).

3

Error conditions:

(3.1) — `resource_unavailable_try_again` — if some non-memory resource limitation prevents initialization.

(3.2) — `operation_not_permitted` — if the thread does not have the privilege to perform the operation.

```
constexpr ~condition_variable_any();
```

4 *Preconditions*: There is no thread blocked on `*this`.

[*Note 2*: That is, all threads have been notified; they can subsequently block on the lock specified in the wait. This relaxes the usual rules, which would have required all wait calls to happen before destruction. Only the notification to unblock the wait needs to happen before destruction. Undefined behavior ensues if a thread waits on `*this` once the destructor has been started, especially when the waiting threads are calling the wait functions in a loop or using the overloads of `wait`, `wait_for`, or `wait_until` that take a predicate. — *end note*]

```
constexpr void notify_one() noexcept;
```

5 *Effects*: If any threads are blocked waiting for `*this`, unblocks one of those threads.

```
constexpr void notify_all() noexcept;
```

6 *Effects*: Unblocks all threads that are blocked waiting for `*this`.

32.7.5.2 Noninterruptible waits [thread.condvarany.wait]

```
template<class Lock>
  constexpr void wait(Lock& lock);
```

1 *Effects*:

(1.1) — Atomically calls `lock.unlock()` and blocks on `*this`.

(1.2) — When unblocked, calls `lock.lock()` (possibly blocking on the lock) and returns.

(1.3) — The function will unblock when signaled by a call to `notify_one()`, a call to `notify_all()`, or spuriously.

2 *Postconditions*: `lock` is locked by the calling thread.

3 *Throws*: Nothing.

4 *Remarks*: If the function fails to meet the postcondition, `terminate()` is invoked ([except.terminate]).

[*Note 1*: This can happen if the re-locking of the mutex throws an exception. — *end note*]

```
template<class Lock, class Predicate>
  constexpr void wait(Lock& lock, Predicate pred);
```

5 *Effects*: Equivalent to:

```
while (!pred())
  wait(lock);
```

```
template<class Lock, class Clock, class Duration>
  constexpr cv_status wait_until(Lock& lock, const chrono::time_-
point<Clock, Duration>& abs_time);
```

6 *Effects*:

(6.1) — Atomically calls `lock.unlock()` and blocks on `*this`.

(6.2) — When unblocked, calls `lock.lock()` (possibly blocking on the lock) and returns.

(6.3) — The function will unblock when signaled by a call to `notify_one()`, a call to `notify_all()`, expiration of the absolute timeout ([thread.req.timing]) specified by `abs_time`, or spuriously.

(6.4) — If the function exits via an exception, `lock.lock()` is called prior to exiting the function.

7 *Postconditions*: `lock` is locked by the calling thread.

8 *Returns*: `cv_status::timeout` if the absolute timeout ([thread.req.timing]) specified by `abs_time` expired, otherwise `cv_status::no_timeout`.

9 *Throws*: Timeout-related exceptions ([thread.req.timing]).

10 *Remarks*: If the function fails to meet the postcondition, `terminate()` is invoked ([except.terminate]).

[*Note 2*: This can happen if the re-locking of the mutex throws an exception. — *end note*]

```
template<class Lock, class Rep, class Period>
  constexpr cv_status wait_for(Lock& lock, const chrono::duration<Rep,
Period>& rel_time);
```

11 *Effects*: Equivalent to:

```
return wait_until(lock, chrono::steady_clock::now() + rel_tim
e);
```

12 *Postconditions*: `lock` is locked by the calling thread.

13 *Returns*: `cv_status::timeout` if the relative timeout ([thread.req.timing]) specified by `rel_time` expired, otherwise `cv_status::no_timeout`.

14 *Throws*: Timeout-related exceptions ([thread.req.timing]).

15

Remarks: If the function fails to meet the postcondition, `terminate` is invoked ([except.terminate]).

[*Note 3*: This can happen if the re-locking of the mutex throws an exception. — *end note*]

```
template<class Lock, class Clock, class Duration, class Predicate>
  constexpr bool wait_until(Lock& lock, const chrono::time_point<Clock,
Duration>& abs_time, Predicate pred);
```

16 *Effects*: Equivalent to:

```
while (!pred())
  if (wait_until(lock, abs_time) == cv_status::timeout)
    return pred();
return true;
```

```
template<class Lock, class Rep, class Period, class Predicate>
  constexpr bool wait_for(Lock& lock, const chrono::duration<Rep,
Period>& rel_time, Predicate pred);
```

19 *Effects*: Equivalent to:

```
return wait_until(lock, chrono::steady_clock::now() + rel_tim
e, std::move(pred));
```

32.7.5.3 Interruptible waits [thread.condvarany.intwait]

1 The following wait functions will be notified when there is a stop request on the passed `stop_token`. In that case the functions return immediately, returning `false` if the predicate evaluates to `false`.

```
template<class Lock, class Predicate>
  constexpr bool wait(Lock& lock, stop_token stoken, Predicate pred);
```

2 *Effects*: Registers for the duration of this call `*this` to get notified on a stop request on `stoken` during this call and then equivalent to:

```
while (!stoken.stop_requested()) {
  if (pred())
    return true;
  wait(lock);
}
return pred();
```

4 *Postconditions*: `lock` is locked by the calling thread.

5 *Throws*: Any exception thrown by `pred`.

6 *Remarks*: If the function fails to meet the postcondition, `terminate` is called ([except.terminate]).

[*Note 2*: This can happen if the re-locking of the mutex throws an exception. — *end note*]

```
template<class Lock, class Clock, class Duration, class Predicate>
  constexpr bool wait_until(Lock& lock, stop_token stoken,
                  const chrono::time_point<Clock, Duration>& abs_time,
Predicate pred);
```

7 *Effects*: Registers for the duration of this call `*this` to get notified on a stop request on `stoken` during this call and then equivalent to:

```
while (!stoken.stop_requested()) {
  if (pred())
    return true;
  if (wait_until(lock, abs_time) == cv_status::timeout)
    return pred();
}
return pred();
```

10 *Postconditions*: `lock` is locked by the calling thread.

11 *Throws*: Timeout-related exceptions ([thread.req.timing]), or any exception thrown by `pred`.

12 *Remarks*: If the function fails to meet the postcondition, `terminate` is called ([except.terminate]).

[*Note 5*: This can happen if the re-locking of the mutex throws an exception. — *end note*]

```
template<class Lock, class Rep, class Period, class Predicate>
  constexpr bool wait_for(Lock& lock, stop_token stoken,
                const chrono::duration<Rep, Period>& rel_time,
Predicate pred);
```

13 *Effects*: Equivalent to:

```
return wait_until(lock, std::move(stoken), chrono::steady_clo
ck::now() + rel_time,
                  std::move(pred));
```

Feature test macro

15.11 Predefined macro names [cpp.predefined]

```
__cpp_constexpr_synchronization 20????L
```

17.3.2 Header <version> synopsis [version.syn]

```
#define __cpp_lib_constexpr_mutex 20????L                    // also in <mutex>
#define __cpp_lib_constexpr_recursive_mutex 20????L          // also in <mutex>
#define __cpp_lib_constexpr_timed_mutex 20????L              // also in <mutex>
#define __cpp_lib_constexpr_timed_recursive_mutex 20????L    // also in <mutex>
#define __cpp_lib_constexpr_shared_mutex 20????L             // also in <shared_mu
tex>
#define __cpp_lib_constexpr_shared_timed_mutex 20????L       // also in <shared_mu
tex>
#define __cpp_lib_constexpr_lock_guard 20????L               // also in <mutex>
#define __cpp_lib_constexpr_scoped_lock 20????L              // also in <mutex>
#define __cpp_lib_constexpr_unique_lock 20????L              // also in <mutex>
#define __cpp_lib_constexpr_shared_lock 20????L              // also in <mutex>
#define __cpp_lib_constexpr_call_once 20????L                // also in <mutex>
#define __cpp_lib_constexpr_locking_algorithms 20????L       // also in <mutex>
#define __cpp_lib_constexpr_condition_variable 20????L     // also in <condition_v
ariable>
#define __cpp_lib_constexpr_condition_variable_any 20????L // also in <condition_v
ariable>
```